



Introduction to python

Introduction to python (Universiteit van Amsterdam)



Scan to open on Studocu

Python week 1:

Python basics:

Python script: text files containing a list of python commands and is a document ending with .py

You use print() to generate output from script

You can insert comments using #

Exponentiation: **. This operator raises the number to its left to the power of the number to its right. For example $4 * 2$ will give 16.

Modulo: %. This operator returns the remainder of the division of the number to the left by the number on its right. For example $18 \% 7$ equals 4.7

Variables and types

You can give variables a value

Type() gives the type of variable

Float() real number → integer/fractional part

Int() integers

String = represent text

Boolean = true or false

Lists

Lists are a method to assign a collection of values to a variable

Lists can contain lists -> sublists

Index to access lists

First element in the list has index 0

[0, 1, 2, 3, 4, 5]

Negative indexing -> other way around

Slicing getting a part of the list [start: end] (start is inclusive and end is exclusive)

List manipulation:

Change lists

Adding elements: lists + [x, y]

Del to delete elements in a list

Functions

Piece of reusable code that solves a specific task

Round(nr, decimal places) -> automatically rounds with only 1 input

Methods:

Functions that belong to objects

Week 2:

Dictionaries:

{key: value}

Print(Europe['norway']) to access the value of the key

Immutable keys (cannot be changed after they have been created)

List is mutable

Add to a dictionary: europe['france'] = 'paris'

Del to delete pairs

Comparison operators:

Comparator	Meaning
<	Strictly less than
<=	Less than or equal
>	Strictly greater than
>=	Greater than or equal
==	Equal
!=	Not equal

If, elif, else:

While loop:

Repeating action until condition is met

Error = 50

While error > 1:

Error = error / 3

Print(error)

For loop:

For var in seq:

Expression

Enumerate produces index and value itself

Loops in dictionary:

For key, value in world.items()

Print(key + "... " + str(value))

Week 3:

User defined functions:

Def square ():

New_value = 4**2 → function body

Print (new_value)

Def square (value):

New_value = value ** 2

Print(new_value)

Tuples:

- Like a list
- Immutable
- Constructed using parentheses()
- You can do indexing

Scope part of the program where an object is accessible

- Global scope: defined in the main body
- Local scope: defined inside a function
- Built-in scope: names pre-defined in python

Nested function:

Functions defined within another function

Default and flexible arguments:

Flexible argument = *args

**kwargs

Lambda functions:

Introduction to error handling:

Use 'except' to handle errors

Week 3:

Variables defined within a function are local to that function: they are inaccessible after a function exists

Global variables (=defined outside every function) are accessible from everywhere

Nested functions:

You can define a function inside another function

Lambda functions:

Function = lambda argument1, argument2: one expression

Print(function(1, 2))

Map() function:

Zip creates an iterable collection of tuples out of two separate iterable collections

Week 6:

Strings: sequence of characters

If a quote is part of the string, you need to use the other type quote to enclose in the string

String operations:

.lower() to lowercase letters

.uppercase() to uppercase letters

.capitalize() to first character in uppercase

.split(sep = ' ', maxsplit = x)

/n = newline

`/r` = carriage return

`.splitlines()` will split in 2

`separator.join(list)` joins a list or iterables together

`.rstrip('char')`

Week 6 lecture:

Basic string manipulation:

To use quotes inside a string, you have to use the other type of quotes:

`"I'm Fay"`

Or use a backslash before the quote to make sure python takes it as a regular character

`"I'm \"tired\""`

Or use concatenation

`\n` to initiate a new line

`"First line. \nSecond line."`

`Reverse = string[::-1]`

`len()` determines the length of the string

`str()` to convert an object into a string

`.upper()` to convert to uppercase

`.lower()` to convert to lowercase

`.capitalize()` to capitalize the first letter

`.split(sep=None, maxsplit = -1)` splits a string into a list

With what you want to join(e.g. whitespace).`join(list)`

`.count(object, start, end)` counts the amount of this object

`.index(object)` returns the index of this object

If the object is not found, it returns an error

`.find(substring, start, end)`

If the object is not found, it returns -1

`.replace(what you want to replace, what you want to replace it with, how many times you want to replace)`

F-strings:

```
Print(f"my {course} midterm grade was {grade}.")
```

Object-oriented programming (OOP)

Objects = data + behavior

is a method of structuring a program by bundling related properties and behaviors into individual objects.

String formatting:

Insert a custom string or variable

positional formatting: 'text {}'.format(value)

str.format()

replaces the placeholder with values, placeholders can hold order numbers so that the code know in which order the place holders need to be filled

Named placeholders so that you can fill them using a dictionary {dictionary[key]}

Format specifier: specify data type to be used {index:specifier}

Datetime

!s string conversion

!r string containing printable representation

!a

You can also use format specifiers

Escape sequences: backslashes \ to include quotes, but this is not allowed in f strings

Template strings

From string import Template

String = Template('Data science hase been called \$identiefier')

String.substitute(identifier='sexiest job')

Use \${identifier} when characters follow the identifier

OOP

Object = state (attributes, variables)+ behavior (methods, functions)

Has a label and triggers an action

Classes are like blue prints, describe states and behaviors that an object can have

Class <name> starts a class definition :

```
Def identify(self, name):  
    Print("I am customer" + name)
```

Attributes are created by assignment

Class customer:

```
Def set_name(self, new_name):  
    Self.name = new_name # will create .name when set_name is called
```

Cust = customer()

__init__ can take multiple attributes

Lecture 7: Data analysis with pandas

What is pandas?

- Built around/on top of Numpy arrays
- Adds labels to rows ('index') and columns ('columns')

Pip install pandas

Import pandas as pd

Df = pd.read_csv(file)

Pd.index

- Represents ordered collection of row and/or column labels

Pd.series

- 1d data with labels

Pd.DataFrame

- 2d data with row labels and column labels

Creating a series through pd.series()

You can specify a different index using index keyword argument

Creating a DataFrame through pd.DataFrame()

You can specify different index and columns using index and columns keyword arguments

Creating an index through `pd.index()`

Viewing and sorting data:

`Df`,

`df.t` = transposing a matrix

`df.head()` = first five rows

`df.tail()` = last five rows

`df.describe()` = get a bunch of descriptions

`df.sort_index()`

- `Axis = 0` rows, `axis 1` = columns
- `Ascending = True/False`

`Df.sort_values()`

Selecting columns:

`Price = csv.price`

Or

`Csv['price']`

Selection by label: series

`s.loc[labels]`

labels can be:

- a list of labels
- a slice of labels
- a boolean series
- a single label to access a scalar element

`df.loc[index_labels, column_labels]`

`df[column_labels]`

- yields a DataFrame for a list of column labels
- yields a series for a scalar column label

`df.column label`

`df.iloc[row_index, column index]`

`df.iat[row_index, column_index]`

Boolean indexing:

Specify a condition on a series

This creates a series with true or false values

Setting values

Add a new dataframe column: `df[column_label] = value`

Missing data: `np.nan`

To indicate a missing value: `df.isna()`

Replace missing data with a value: `df.fillna(value)`

Drop rows/columns with missing data: `df.dropna()`

Basic operations

Max, mean, etc

Mapping a function to series elements

`s.map(function_name)`

`s.map(lambda x: ..)`

Vectorized string methods

We may want to apply the same string method, indexing or slicing to every element

`s.str[::-1]`

`s.str.lower()`

Applying a function to each row or to each column

`Df.apply(function_name, axis = 0)`

`Df.apply(lambda x: , axis = 0)`

Lecture 8:

List comprehension:

X = [expression for running_variable in iterable if condition]

➔ Otherwise long for loop

[letter.upper() if letter.islower() else letter.lower() for letter in "AbC"]

Dictionary comprehension:

X = {key_name: expression for running_variable in iterable if condition}

Set comprehension:

Generators are iterable objects, potentially representing an arbitrarily large collection of values without memory problems

(expression for running_variable in iterable if condition)

Generators exhaust

Generators create a next and next

Handling dates and times

➔ Datetime package in the Python standard library

From datetime import datetime

D = datetime(2022, 5, 24)

Print(d)

2022-05-24 00:00:00

Print(d.year, d.month, d.day)

2022 5 24

Formatting datetime values (strftime)

%d: day of the month

%m: month

%Y: year

Print(d.strftime("%m/%d?%Y"))

05/24/2022

```
D_US = datetime.strptime(s, "%m/%d/%Y")
```

P = parsing a string

```
From datetime import timedelta
```

```
D = datetime(2022, 5, 24)
```

```
Diff = timedelta(days=10)
```

```
Print(d + diff)
```

```
Print(type(d + diff))
```

Aware vs naïve datetime objects

Datetime objects also record hours, minutes, seconds and microseconds

“Aware” : with time zone information

“Naïve”: no time zone information

```
Import pytz
```

```
Utc = pytz.timezone("UTC")
```

```
Ams = pytz.timezone("Europe/Amsterdam")
```

```
D = datetime(2022, 5, 24, 10, 0, tzinfo=utc)
```

```
Print(d)
```

```
2022-05-24 10:00:00+00:00
```

```
Print(d.astimezone(ams))
```

```
2022-05-24 12:00:00+02:00
```

Datetime.now() tells you what time it is now

Week 8 datacamp:

Iterables: lists, strings, dictionaries, file connections

An object with an associated `iter()` method

Iterator produces the next()

Creating an iterator using: `iter()`

`.strptime()` converts a string to a datetime object

`.strftime()` uses a format string to convert a datetime object to a string

Timedelta is used to represent an amount of change in time

`.parse()` will attempt to convert a string to a pendulum datetime object without the need of the format string